

LDPC Codes Implementation

Large (204,104) LDPC Experiment Report

Course: Information Theory

Author: Faisal Iqbal

Language: Python

Experiment configuration: N=204, K=104, M=100, R=0.5098, frames per point=300

2. Abstract

This report presents a large-block LDPC experiment for a Gallager-style (204,104) code [1] with 100 parity checks and actual rate $R=0.5098$. The study evaluates transmission over the Binary Symmetric Channel, Additive White Gaussian Noise channel, and Binary Erasure Channel. Decoder performance is measured using bit error rate, frame error rate, convergence rate, average iteration count, and coding gain relative to uncoded baselines.

Artifact Note

Some expected experiment artifacts were not found when this report was generated. Run `python experiment.py` to regenerate the latest figures, CSV tables, and summary JSON. Missing artifacts:

- results\figures\bsc_ber_fer.png
- results\figures\awgn_ber_fer.png
- results\figures\bec_ber_fer.png
- results\figures\decoder_convergence.png
- results\figures\coding_gain_summary.png
- results\tables\bsc_results.csv
- results\tables\awgn_results.csv
- results\tables\bec_results.csv

3. Introduction

Low-Density Parity-Check codes were introduced by Gallager [1] as sparse linear block codes decoded by iterative message passing on a Tanner graph. Their practical value comes from sparse parity constraints and soft-information decoders such as Belief Propagation and Min-Sum, as commonly presented in coding theory texts [2], [3]. This report focuses on the large experiment used by the project workflow and does not include toy-code demonstrations.

4. LDPC Code Construction

The experiment targets length 200 and resolves to the nearest valid Gallager construction [1] length $N=204$. The resulting parity-check matrix has $M=100$ rows and rank 100 over $GF(2)$, giving actual dimension $K=104$ and rate 0.5098. The report does not print full H or G matrices; instead it summarizes their structural properties.

5. Encoding and Channel Models

Encoding uses a generator matrix obtained from $GF(2)$ row reduction of the parity-check matrix. Messages of length K are mapped to length- N codewords satisfying $Hc = 0$ over $GF(2)$. The channel models are BSC with crossover probability p , AWGN with BPSK modulation and E_b/N_0 sweeps, and BEC with erasure probability ϵ , following standard information-theoretic channel models [2], [4]. Uncoded baselines are included so that coding gain can be computed at each simulated point.

6. Decoding Algorithms

The BSC experiment evaluates Bit-Flip, Belief Propagation, and Min-Sum decoding [2], [3]. The AWGN experiment evaluates BP and Min-Sum using soft LLRs. The BEC experiment evaluates BP and Min-Sum only through the supported LLR decoder path; if their curves coincide, the report treats that as an observed equality rather than as evidence of different behavior. BP and Min-Sum use a maximum of 50 iterations, while Bit-Flip uses 15 iterations.

7. Experimental Methodology

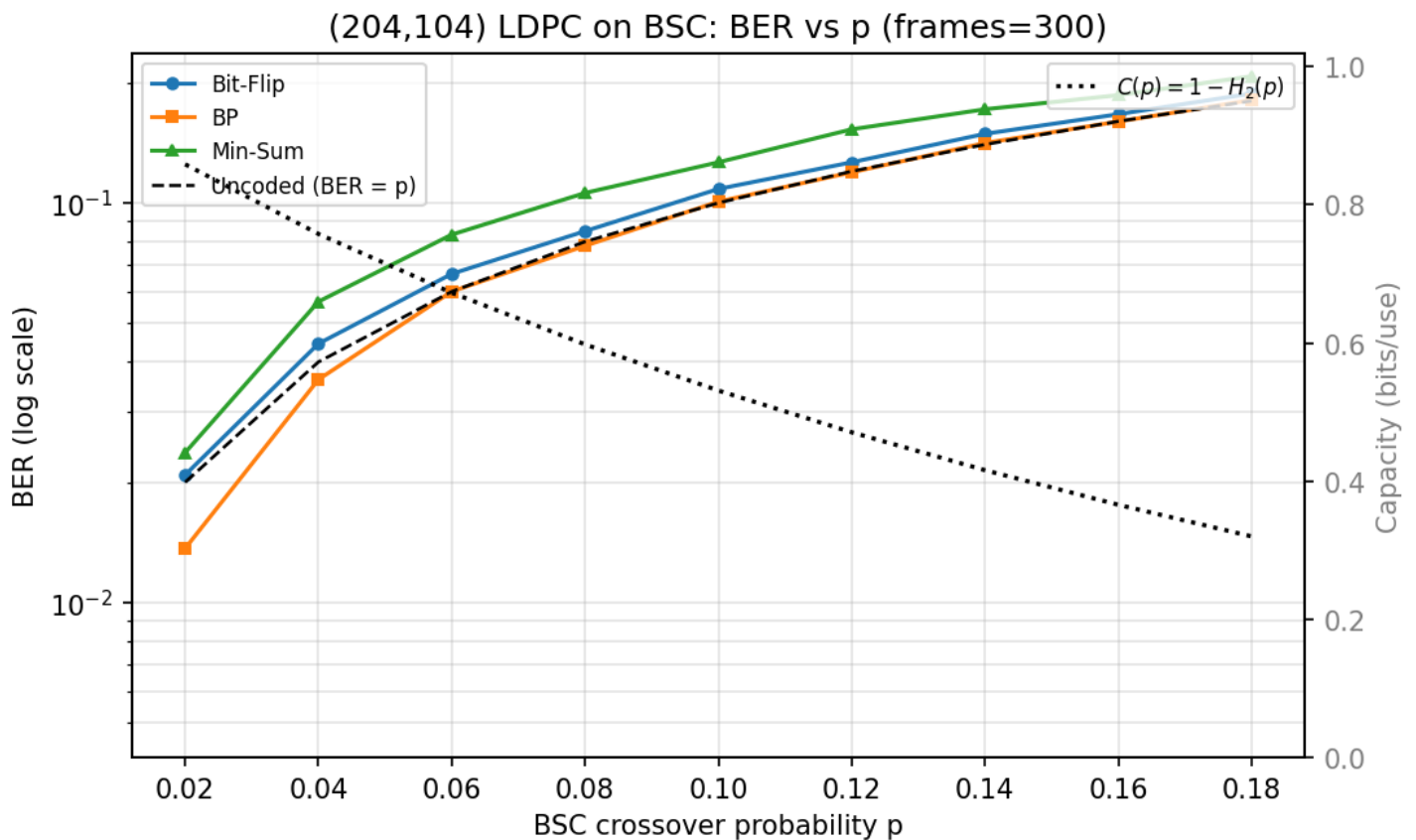
Each channel point uses 300 Monte Carlo frames by default. For every channel point and decoder, the experiment records uncoded BER, decoder BER, decoder FER, convergence rate, average iterations, and coding gain in dB. Coding gain is computed directly as $10 \log_{10}(\text{uncoded_BER} / \text{decoder_BER})$. The use of uncoded baselines follows the comparison style used in information theory and coding experiments [2], [4]. Negative values are retained because they indicate that the coded configuration performed worse than the uncoded baseline at that point.

```
python experiment.py
python generate_report.py
```

8. Results and Discussion

BSC

BSC BER/FER



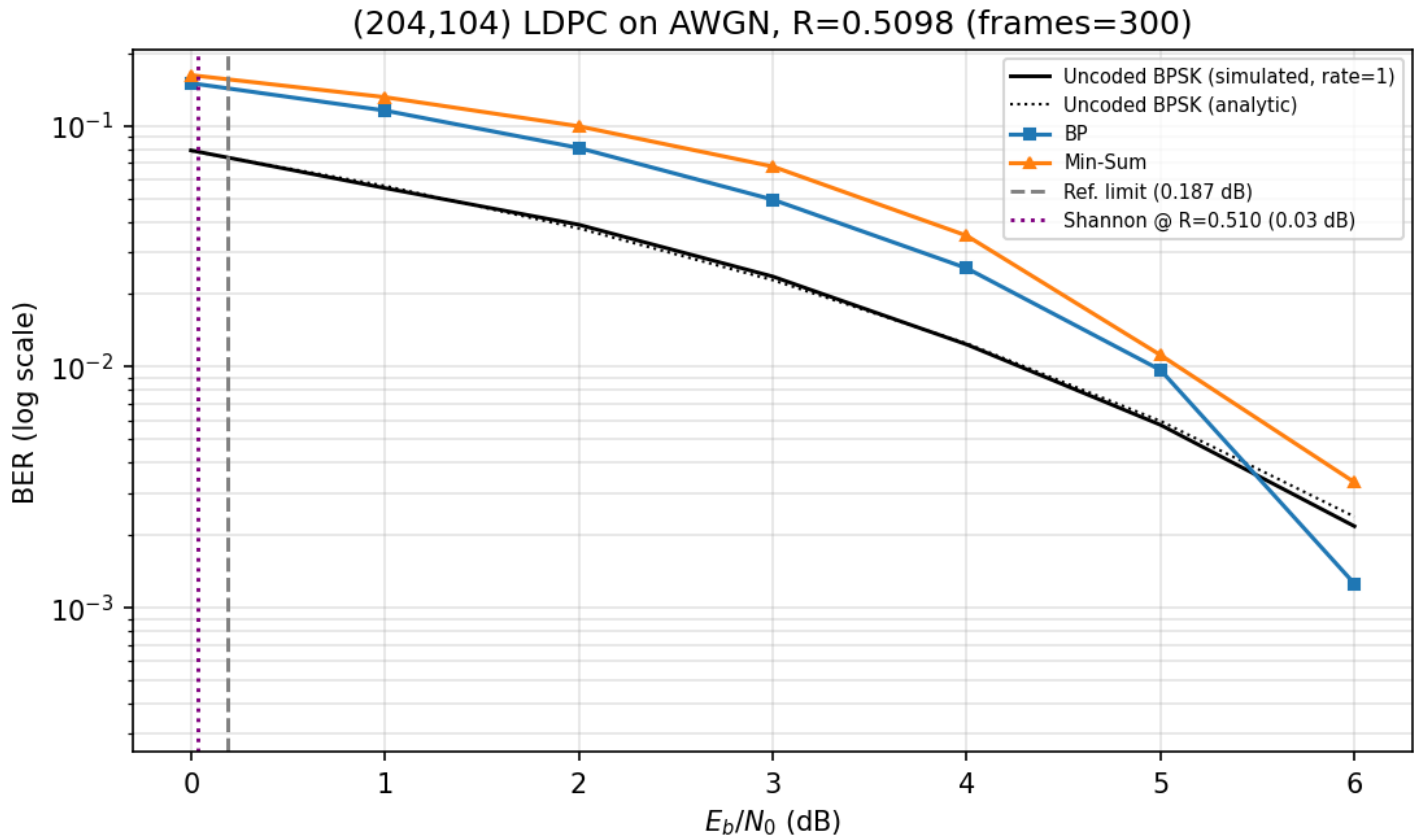
Note: expected results\figures\bsc_ber_fer.png was not available, so the existing figure results\figures\ber_bsc_with_capacity.png was used as a fallback.

BSC Numeric Summary

No CSV table was available for BSC. Expected file: results\tables\bsc_results.csv.

The BSC results compare the hard-decision Bit-Flip decoder with soft-decision BP and Min-Sum. The uncoded baseline is shown explicitly. The table should be used to identify any points where the coded BER is worse than uncoded, rather than assuming uniform improvement.

AWGN

AWGN BER/FER

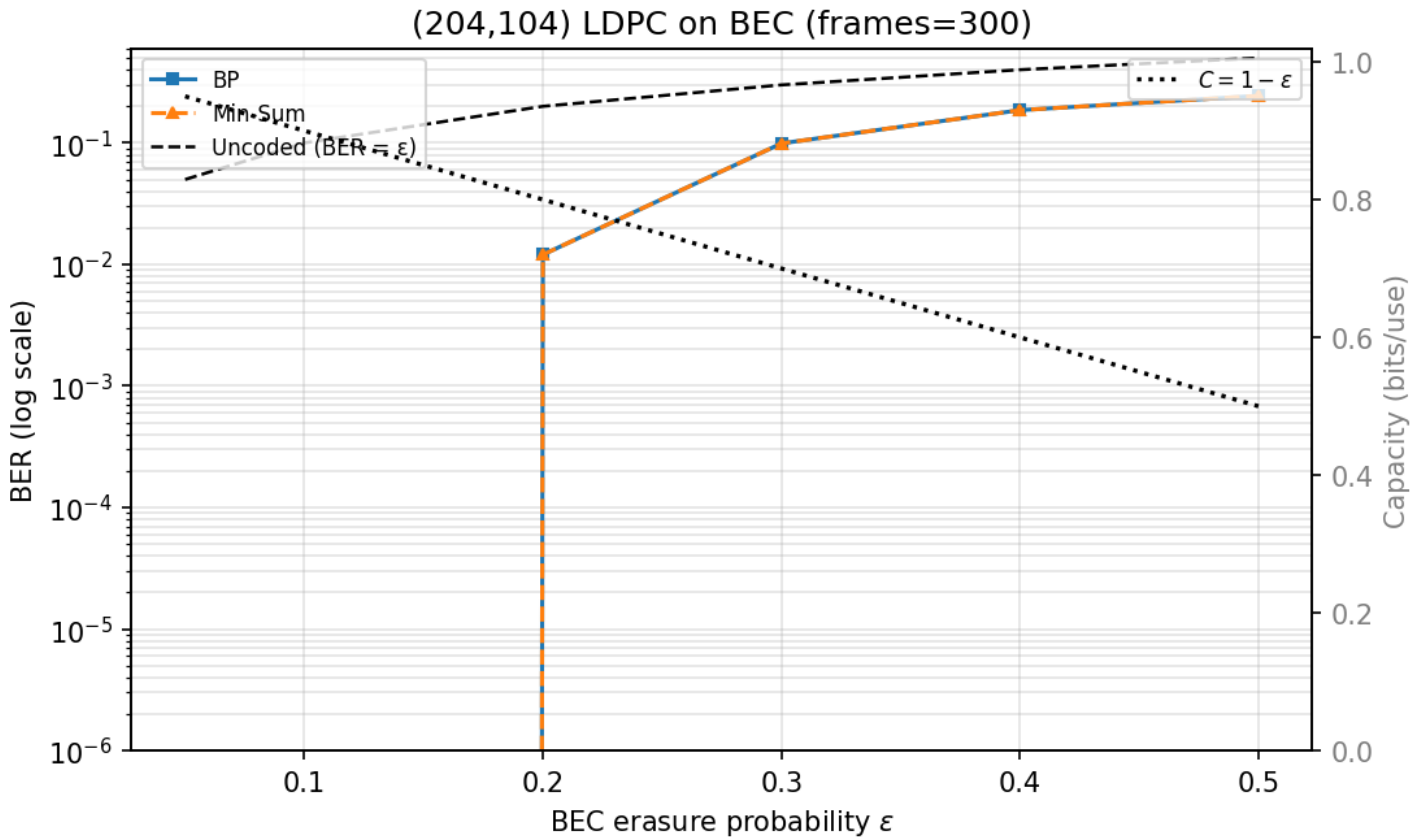
Note: expected results\figures\awgn_ber_fer.png was not available, so the existing figure results\figures\ber_awgn_with_shannon.png was used as a fallback.

AWGN Numeric Summary

No CSV table was available for AWGN. Expected file: results\tables\awgn_results.csv.

The AWGN results use E_b/N_0 as the sweep variable. Shannon references [4], when present in the figure, are theoretical markers only. They are not claims that the finite-length implementation approaches capacity. Negative coding gain can occur at low or poorly matched operating points.

BEC**BEC BER/FER**



Note: expected results\figures\bec_ber_fer.png was not available, so the existing figure results\figures\ber_bec_with_capacity.png was used as a fallback.

BEC Numeric Summary

No CSV table was available for BEC. Expected file: results\tables\bec_results.csv.

The BEC results should be interpreted carefully when BP and Min-Sum coincide. Matching values are reported as matching values and should not be described as separate performance behavior.

Coding Gain

Coding Gain Summary

Figure not available: results\figures\coding_gain_summary.png. Run python experiment.py to generate it.

Coding gain is signed. Positive values indicate improvement relative to the uncoded baseline, while negative values indicate the coded decoder performed worse at that channel point. This report therefore describes each channel point according to the measured table values.

Convergence Behavior

Decoder Convergence

Figure not available: results\figures\decoder_convergence.png. Run python experiment.py to generate it.

Convergence rate and average iteration count provide context for BER and FER. A decoder may show acceptable BER at some points while still requiring many iterations or failing to converge often at more difficult channel settings.

Validation Warnings

The experiment summary includes the following warnings. They are intentionally preserved:

- AWGN: Min-Sum BER not strictly below uncoded (sim.) at indices [0, 1, 2, 3, 4, 5, 6]

9. Limitations

The implementation is an educational Python implementation rather than an optimized LDPC simulator. The block length is finite, the construction may contain short cycles, and the default Monte Carlo budget is modest. Results therefore show the behavior of this specific code and decoder setup; they should not be generalized as near-capacity performance [3], [4].

10. Conclusion

The project implements and evaluates a large (204,104) LDPC code across BSC, AWGN, and BEC channel models. The current workflow records both reliability metrics and diagnostic metrics, including FER, convergence rate, average iterations, and signed coding gain. The most important interpretation is empirical: the generated CSV tables and summary JSON determine whether coding gain is positive or negative at each channel point.

11. References

- [1] Gallager, R. G., Low-Density Parity-Check Codes, MIT Press, 1963. <https://mitpress.mit.edu/9780262570940/low-density-parity-check-codes/>
- [2] MacKay, D. J. C., Information Theory, Inference, and Learning Algorithms, Cambridge University Press, 2003. <https://www.inference.org.uk/itila/book.html>
- [3] Richardson, T. and Urbanke, R., Modern Coding Theory, Cambridge University Press, 2008. <https://doi.org/10.1017/CBO9780511791338>
- [4] Shannon, C. E., A Mathematical Theory of Communication, Bell System Technical Journal, 1948. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>